

2026 Computer Graphics Project 3 Report

Name: MinYong Seo, 서민용

Student ID: 2022056562

1. Video Link: <https://youtu.be/eFk1LP9Kw3g>

2. Core Motion Viewer

A. Explain how your viewer visualizes skeletal motion and why you chose that representation

- 우선 BVH 파일을 parsing해서 각 joint에 대한 offset 값들과 frame에 따른 각 channel들의 값들을 얻었습니다.
 - <http://motion.hahasoha.net/> 에서 파일을 받아보았는데, 구현하고자 하는 동작에 대해서 크게 필요없는 정보들, 예를 들어서 hand에 대해서는 손가락 데이터들, head에 대해서는 눈 데이터들 등 크게 쓸모가 없는 것들은 parsing 할 때 제외했습니다.
- 스켈레톤을 표현할 때는 joint는 sphere로, joint간의 연결부는 cylinder를 사용했습니다.
 - 그 이유는 project2에서 lighting을 구현했는데, cube를 사용할 경우 lighting 효과가 덜 드러날 것 같다고 생각해서 입니다.
 - 물론 이번 project3는 lighting이 주안점은 아니라서 lighting에 대한 추가적인 기능이 없지만 조금 아깝다고 생각했습니다.
- 스켈레톤의 구조를 class로 나타낼 때, lab과 비슷하지만 조금 다른 방식을 사용했습니다.
 - BVHJoint가 기존의 hierarchical 구조를 가지고, joint로써 joint transformation과 link transformation을 수행합니다.

```
class BVHJoint:
    def __init__(self, name, parent=None):
        self.name = name
        self.parent = parent
        self.children = []
        self.offset = glm.vec3(0)
        self.channels = []
        self.ch_start = 0
        self.is_end = False

        self.link_transform = glm.mat4(1.0)
        self.joint_transform = glm.mat4(1.0)
        self.global_transform = glm.mat4(1.0)
```

- 하나 기존 lab과의 차이점은, 각 frame 마다 얼마나 회전할 것인가를 BVH data를 통해서 알게 되기 때문에, 각 스켈레톤들은 BVHMotion으로써 BVHJoint들을 가지고 data를 넣어가며

```
class BVHMotion:
    def __init__(self):
        self.root = None
        self.joints = []
        self.n_frames = 0
        self.frame_time = FRAME_DT
        self.frames = []
```

조종한다고 볼 수 있습니다.

- 물론 hierarchical model 로써의 기능이 있기 때문에, 매 frame 별로 draw 하는것과 별개로, 각 joint들의 transformation matrix는 매 frame 별로 root를 업데이트 시키는 것으로 구현됩니다.

```
def apply_frame(self, frame_idx):
    frame_idx = max(0, min(frame_idx, self.n_frames - 1))
    data = self.frames[frame_idx]
    for j in self.joints:
        j.set_joint_transform(data)
    self.root.update_tree_global_transform()
```

B. Explain your interaction design (keyboard, mouse, etc.) and why you designed it that way

- 기존 project1과 project2와 동일하게, 기본적으로는 사용자의 입력에 따라 Orbit 카메라 시스템으로 움직일 수 있습니다.
- 또한 project2와 비슷하게, 특정 키를 누르면 cam_center가 특정 object의 위치와 같게하여 tracking 하거나, 스켈레톤의 시점으로 카메라를 고정시킬 수 있습니다.
 - 1,3 키를 누를 경우 각각 투수와 타자의 위치로 cam_center 가 고정되어 투수와 타자를 중심으로 관찰할 수 있습니다.
 - 2,4 키를 누를 경우, 각각 투수와 타자의 head 위치와 head가 바라보는 시점으로 카메라가 고정됩니다.
 - F 키를 누를 경우 다시 기존 사용자의 Orbit camera로 복귀합니다.
- 이번 project의 특색으로 구현한 기능은 바로 K키를 누를 경우 슬로우모션과 함께 각각 투수와 타자의 동작을 면밀히 볼 수 있는 기능입니다.
 - K키를 한번 누르면 투수에게 카메라가 고정되고, 자동으로 카메라가 투수를 기준으로 orbit 합니다. 또한 시간이 느리게 흐르게 합니다.
 - K키를 한번 더 누르면 타자에 대해 위와 똑같은 동작이 일어납니다.
 - K키를 한번 더 누르면 기존 카메라 모드로 돌아갑니다.
 - 참고로 슬로우모션 모드에서 1,2,3,4,F를 눌러도 해당 카메로 모드로 가지 않고, 오로지 K를 3번 눌러서 복귀할 수 있습니다.
- 또한 애니메이션이 반복되도록 구현하였는데, 이와 관련하여 R키를 누르면 즉시 다시 애니메이션을 재생할 수 있습니다.

C. Specify the BVH files used in your project, including the source dataset name and a brief description of the motion (e.g., walking, dancing, martial arts, etc.)

- <http://motion.hahasoha.net/> 에서 각각 pitch motion과 swing motion인 124_01.bvh와 124_07.bvh를 사용했습니다.
- 조금의 특이사항은, 124_07.bvh가 2번 스윙하는 데이터 파일이기 때문에, 타자를 렌더링할 때 max_frame을 두어서 딱 한번만 스윙하도록 코드 레벨에서 제한을 두었습니다.

3. Motion Application / Exploration

A. Explain what motion-related idea or application you explored

- 투수와 타자에 대한 것이기 때문에, 배트와 공이 빠질 수 없었습니다.
- 하여 투수와 타자는 각각 배트와 공을 들고 있고, 일정시간이 지나면 공이 투수의 손을 떠나서 던져지는 것처럼 보이도록 구현했습니다.
 - 이에 대해서 교수님께서 전에 수업중에 인터넷에서 찾으신 중세기사 모델링에 대해서, "물론 칼을 떨어트리는 건 구현이 없기 때문에 그냥 손의 child로써 존재하는 것이다."라는 말씀을 하신 게 기억이 나서,
배트는 이와 비슷하니 구현해볼 수 있겠다는 생각이 들었습니다.
 - 또한 공도 역시 투수의 손에 붙어있다가, 일정 시간이 되면 날아가는 것처럼 구현한다면 실제로 투수가 공을 던지는 것처럼 만들 수 있겠다는 생각이 들었습니다.

B. Explain how your exploration/application works

- 우선 배트와 공에 대해서 공통적으로 쓰이는 함수가 있습니다.
 - BVHMotion 내부 함수인 joint_by_name으로, name을 알았을 때 해당 BVHJoint를 얻을 수

```
def joint_by_name(self, name):  
    for j in self.joints:  
        if j.name == name:  
            return j  
    return None
```

있게합니다.

- 공 같은 경우엔, 투수의 모션에 따라서 몇 frame쯤 될 때 던져지는지 대략적으로 파악하여, [하드코딩으로 돌려가면서 최적의 값을 찾았습니다] 이를 고정값으로 사용하고, 이를 bvh 파일에 나와있는 frame time과 곱해서 시간으로 구해둡니다.

```
BALL_RELEASE_T = BALL_RELEASE_FRAME * FRAME_DT
```

- while 문에 들어가 렌더링을 시작하기 전, 미리 위의 코드를 활용하여 투수의 오른손 joint를 알아내고, 이를 통해 world frame 내에서 어느 위치인지 파악합니다.

- 또한 타자의 위치 부근에 스트라이크 존을 설정해두어, 공이 타자의 손을 떠나는 위치에서 스트라이크 존으로 날아갈 수 있도록 방향을 구합니다.

```
pitcher_motion.apply_frame(BALL_RELEASE_FRAME)
rhand_joint = pitcher_motion.joint_by_name('rHand')
release_pos = (get_world_pos(rhand_joint, pitcher_wmat)
               if rhand_joint else glm.vec3(0, 1, 0)) + BALL_HAND_OFFSET
strike_zone = BATTER_POS + glm.vec3(0, 1.1, 0)
ball_dir = glm.normalize(strike_zone - release_pos)
```

- 이후 렌더링 시 시각에 따라서 공의 위치를 결정할 수 있습니다.

```
def get_ball_pos(t, rhand_now, release_pos, ball_dir):
    if t < BALL_RELEASE_T:
        return rhand_now
    dt = t - BALL_RELEASE_T
    return glm.vec3(
        release_pos.x + ball_dir.x * BALL_SPEED * dt,
        release_pos.y,
        release_pos.z + ball_dir.z * BALL_SPEED * dt,
    )
```

- 이 공의 위치를 토대로 draw 해주면 됩니다. 중력은 고려하지 않고 그냥 간단하게 직선으로 날아가는 것으로 구현했습니다..
- 배트 같은 경우엔, 그냥 타자의 손에 들려있으면 되기에 구현이 더 간단합니다.

```
bat_local = (glm.rotate(BAT_ROT_X, glm.vec3(1, 0, 0)) *
             glm.translate(glm.vec3(0, BAT_LENGTH * 0.5, 0)) *
             glm.scale(glm.vec3(BAT_RADIUS, BAT_LENGTH * 0.5, BAT_RADIUS)))
```

- 타자의 손에서 어느 위치에 어느 크기만큼 있을지 미리 결정해두고,
- 공과 똑같이 타자의 오른손 joint를 통해서 그려주면 됩니다.

```
rhand_b = batter_motion.joint_by_name('rHand')
M_bat = batter_wmat * rhand_b.global_transform * bat_local
```

C. Explain why you chose that idea or what you found interesting about it

- 기본적으로 야구 중계나 하이라이트에서 아이디어를 얻었습니다
- 추가적으로, 마인크래프트라는 게임에서는 본인의 캐릭터를 움직일 때 1인칭을 사용할 건지 3인칭을 사용할지 결정할 수 있는데 이에 대해서도 아이디어를 얻었습니다
- 또한 1인칭 시점 카메라를 구현해두었기 때문에 손에 배트나 공이 들려있고, 공이 타자를 향해 날라오면 실제 투수와 타자의 시점을 체험하는 효과도 더 커졌다는 생각이 들었습니다
- 또한 난이도 상 공은 단순히 y값은 고정되고, 동일한 속도로 스트라이크 존으로 날아가도록 구현하고, 타자가 헛스윙을 해서 그냥 그대로 뒤로 날아가는 걸로 구현했는데,
 - 이와 관련해서 수업시간에 다른 Physics-Based Animation 왜 강력한가를 몸소 체험한 듯 합니다.

D. Additional Notes (Optional)

- 애니메이션을 시간이 생각보다 많이 짧아서, loop를 이루도록 구현했습니다.
- 또한 Killcam 동작 시 슬로우모션이 들어가야했기 때문에, 실제 glfwGetTime()의 값을 사용하는 것이 아니라, 애니메이션중인 시간에 비율을 적용하는 방식을 사용했습니다.
- 또한 무엇보다 모든 애니메이션은 이 시간에 따라 동작하고, R키를 누르면 즉시 다시 재생되는 로직이 있습니다.
- 하여 차지하는 볼륨이 꽤 큰 로직인데, template에 있는 질문에 마땅히 적을 곳이 없어서 여기에 한번 구현에 대해 작성해보았습니다.

```
now = glfwGetTime()
real_dt = now - g_prev_real
g_prev_real = now

speed = KILLCAM_SPEED if g_killcam_mode > 0 else 1.0
g_anim_t += real_dt * speed
if g_anim_t >= LOOP_DURATION:
    g_anim_t = 0.0
if g_killcam_mode > 0:
    g_killcam_orbit += real_dt * KILLCAM_ORBIT_W
```

- g_anim_t는 전역변수로, 렌더링 시 glfwGetTime을 활용하여 현재 얼마나 지났는지에 대한 정보를 가져와서, 별개로 흐르도록 합니다.
- 만일 K키를 눌러서 슬로우모션이 적용되어야한다면 speed의 값을 1이 아니라 따로 정의해둔 고정값을 사용해서 g_anim_t가 덜 늘어나도록 조절합니다.
- 또한 loop되는 시간을 8초로 따로 정의해두었고, 이 시간이 지나면 다시 0으로 초기화되도록 설정했습니다.
- 이 g_anim_t를 활용해서 투수와 타자의 motion data에서 현재 어느 frame의 데이터를 사용해야하는지 파악하여 이 값으로 렌더링 합니다.

```
p_frame = pitcher_motion.frame_at(g_anim_t)
if g_anim_t < BATTER_DELAY:
    b_frame = BATTER_READY_FRAME
else:
    batter_t = g_anim_t - BATTER_DELAY
    b_frame = min(batter_motion.frame_at(batter_t), BATTER_MAX_FRAME)

pitcher_motion.apply_frame(p_frame)
batter_motion.apply_frame(b_frame)
```